esan UNIVERSITY | Ingeniería de Software

// Bootcamp AWS: de Cloud a Data Lakehouse

De cero a tu primer ETL en en AWS

Cuenta lista, Terraform y un pipeline que reacciona a eventos

Ponente: Ricardo Fabian Uculmana Quispe

Sesión 1 · 5 nov 2026

Agenda

01	Qué es AWS
02	Tu cuenta lista
03	Un bucket, tres formas
04	Del código al pipeline
05	Cierre

Lo que tendrás funcionando hoy

ORIGEN

Archivo NDJSON NDJSON

Una fila JSON por línea

→

S3 · LANDING

Llega el archivo

Lo subes con la AWS CLI CLI

→

LAMBDA

Reacciona sola

Convierte JSON a Parquet Parquet

→

S3 · PROCESSED

Sale el Parquet

Más pequeño y listo para para consultar

AWS en tres ideas

- **Región:** el lugar donde viven tus recursos. En el bootcamp usamos una sola.
- **Servicio:** una pieza que hace una tarea. S3 guarda, IAM da permisos, Lambda ejecuta. ejecuta.
- **API:** todo se pide por API. La consola, la CLI y Terraform solo la llaman.

Regiones y zonas

REGIÓN

Un lugar del mundo

Un conjunto de centros de datos de AWS. Cada recurso vive en una región.

ZONA DE DISPONIBILIDAD

Centros de datos aislados

Cada región tiene varias, para que un fallo no tumbe todo.

Los servicios del bootcamp

SESIÓN 1 · HOY

S3, IAM, Lambda y CloudWatch

Guardar archivos, dar permisos, ejecutar código y ver los logs.

SESIÓN 2 · 12 NOV

Glue, Athena y Step Functions

Transformar, consultar con SQL y coordinar. EventBridge lo programa.

Seguridad compartida

AWS

Seguridad de la nube

Centros de datos, hardware y servicios gestionados.

TÚ

Seguridad en la nube

Usuarios, permisos y credenciales: lo que configuras y configuras y proteges.

Seis pilares para diseñar bien

- **Seis pilares** guían el diseño en AWS: operaciones, seguridad, fiabilidad, rendimiento, rendimiento, costos y sostenibilidad.
- **Hoy practicas dos:** seguridad, con IAM y sin root, y excelencia operativa, con infraestructura como código.
- **Los demás** aparecen cuando el pipeline crece: costos, fiabilidad y rendimiento.

Cuenta nueva: Free Plan o Paid Plan

FREE PLAN

Sin cargos, con fecha de fin

Termina a los 6 meses o al gastar los créditos; entonces la cuenta se cierra.

PAID PLAN

Sin vencimiento, con cargos

Pagas lo que supere los créditos y la cuenta sigue abierta.

02

Tu cuenta lista

No se trabaja con root

ROOT

Dueño de la cuenta

Acceso total. Con MFA y solo para emergencias.

emergencias.

IAM USER

Tu usuario de trabajo

Permisos a medida y una access key para la CLI.

Cuatro conceptos de IAM

USUARIO

Quién eres

Tu IAM user de trabajo

trabajo

→

POLÍTICA

Qué puedes hacer

Hoy: AdministratorAccess

AdministratorAccess

→

ACCESS KEY

Cómo te identifica la CLI

Se muestra una sola vez

vez

→

ROL

Permiso que se pone la Lambda

Lo veremos en el código

código

02 · Tu cuenta lista

Anatomía de una policy

Qué se permite (Effect), qué acción (Action) y (Action) y sobre qué (Resource).

```
# archivo: politica.json (ejemplo)
{
 "Version": "2012-10-17",
 "Statement": [{
  "Effect": "Allow",
  "Action": "s3:GetObject",
  "Resource": "arn:aws:s3:::<landing>/*"
 }]
}
```

Cómo decide IAM

PASO 1

Todo denegado

Sin permiso, nadie puede nada

PASO 2

Un Allow abre

Una política permite la acción

PASO 3

Un Deny gana

Un Deny explícito siempre prevalece prevalece

Creamos tu IAM user



En vivo: crea tu IAM user junto con el ponente.

Configura tu perfil

Un perfil con nombre evita mezclar cuentas. cuentas. La clave secreta se ve una sola vez. vez.

```
# terminal: PowerShell o bash
aws configure --profile bootcamp
AWS Access Key ID: <tu access key id>
AWS Secret Access Key: <tu clave secreta>
Default region name: <región del bootcamp>
Default output format: json

aws sts get-caller-identity --profile bootcamp
aws s3 ls --profile bootcamp
```

Reglas con las credenciales

- **Nunca** en el repositorio, el código, Terraform, el chat ni capturas de pantalla.
- **Siempre** con el perfil nombrado: Terraform lee el perfil, no lleva claves.
- Hay bots que detectan claves expuestas en GitHub en cuestión de minutos.
- AdministratorAccess es solo para este laboratorio: una clave expuesta da control total. control total.

Hoy admin, mañana mínimo privilegio

HOY · LABORATORIO

Admin y access key

Un IAM user con AdministratorAccess y una clave de larga duración.

EN TU EMPRESA

Mínimo privilegio

MFA, permisos justos y credenciales temporales: roles o IAM Identity Center.

Checkpoint: ¿ya funciona?

- El comando get-caller-identity devuelve tu usuario, no root.
- La región configurada es la del bootcamp.
- Tu access key está guardada fuera de carpetas compartidas.

03

Un bucket, tres formas

Tres formas de llamar a la misma API

CLICS

Consola web

Visual, pero no repetible repetible

→

COMANDOS

AWS CLI

Repetible, pero a mano mano

→

CÓDIGO

Terraform

Repetible, versionable y versionable y reversible reversible

→

AWS

La API de AWS

Recibe las tres peticiones peticiones

Bucket 1: la consola



En vivo: crea el primer bucket desde la consola.

Bucket 2: la CLI

El mismo bucket, ahora con un comando. El comando. El nombre es global: usa tu sufijo. sufijo.

```
# terminal: AWS CLI
# 1. crear el bucket (nombre global)
aws s3 mb s3://bootcamp-<sufijo>-cli \
  --profile bootcamp

# 2. listarlo
aws s3 ls --profile bootcamp

# 3. borrarlo
aws s3 rb s3://bootcamp-<sufijo>-cli \
  --profile bootcamp
```

Infraestructura como código

IMPERATIVO

Describes los pasos

Consola y CLI: tú decides el orden y repites los pasos a pasos a mano.

DECLARATIVO

Describes el destino

Terraform: declaras el estado deseado y calcula qué qué cambiar.

Terraform frente a CloudFormation

CRITERIO	CLOUDFORMATION	CDK	TERRAFORM
Lenguaje	YAML o JSON	Python, TypeScript	**HCL**
Nubes	Solo AWS	Solo AWS	**Varias**
Estado	Lo guarda AWS	Lo guarda AWS	**Archivo state**
Ciclo	Plantilla y stack	synth y deploy	**init, plan, apply**

Bucket 3: Terraform

Declaras qué quieres; Terraform decide cómo crearlo.

```
# archivo: main.tf
terraform {
  required_version = ">= 1.6"
}

provider "aws" {
  profile = var.profile
  region  = var.region
}

resource "aws_s3_bucket" "demo" {
  bucket = "bootcamp-${var.sufijo}-tf"
}
```

Anatomía de HCL

Un bloque tiene tipo y nombre; las referencias ordenan el despliegue.

```
# archivo: main.tf (anatomía)
# resource  <tipo>  <nombre local>
resource "aws_s3_bucket" "demo" {
  bucket = "bootcamp-${var.sufijo}-tf"
}

# referencia: tipo.nombre.atributo
resource "aws_s3_bucket_versioning" "v" {
  bucket = aws_s3_bucket.demo.id
  versioning_configuration {
    status = "Enabled"
  }
}
```

Variables y outputs

Las variables parametrizan; los outputs devuelven datos. Aquí no van claves.

```
# archivo: variables.tf y outputs.tf
variable "sufijo" {
  type        = string
  description = "Tus iniciales y 4 dígitos"
}

variable "region" {
  type    = string
  default = "<región del bootcamp>"
}

output "bucket_demo" {
  value = aws_s3_bucket.demo.id
}
```

Cuatro comandos de Terraform

PASO 1

init

Descarga el provider de de AWS

→

PASO 2

plan

Muestra qué va a cambiar

→

PASO 3

apply

Crea los recursos

→

PASO 4

destroy

Elimina lo que creó

Cómo leer un plan

El plan lista qué cambiará antes de tocar tocar nada. Léelo siempre.

```
# terminal: terraform plan
# aws_s3_bucket.demo will be created
+ resource "aws_s3_bucket" "demo" {
    + bucket = "bootcamp-ab1234-tf"
  }

Plan: 1 to add, 0 to change, 0 to destroy.

# + crear      ~ cambiar en el lugar
# - destruir   -/+ destruir y recrear
```

State: la memoria de Terraform

- **Qué es:** terraform.tfstate, un JSON que une tu código con los recursos reales de AWS. de AWS.
- **Para qué:** con él, el plan sabe qué existe, qué cambió y qué falta crear.
- **Reglas:** no se edita a mano ni se sube a Git; puede contener datos sensibles.
- **Inspección:** terraform state list y terraform state show <recurso>.

State local o remoto

LOCAL · HOY

Un archivo en tu equipo

Simple para aprender; se pierde si borras la carpeta.

REMOTO · EQUIPOS

Un bucket de S3

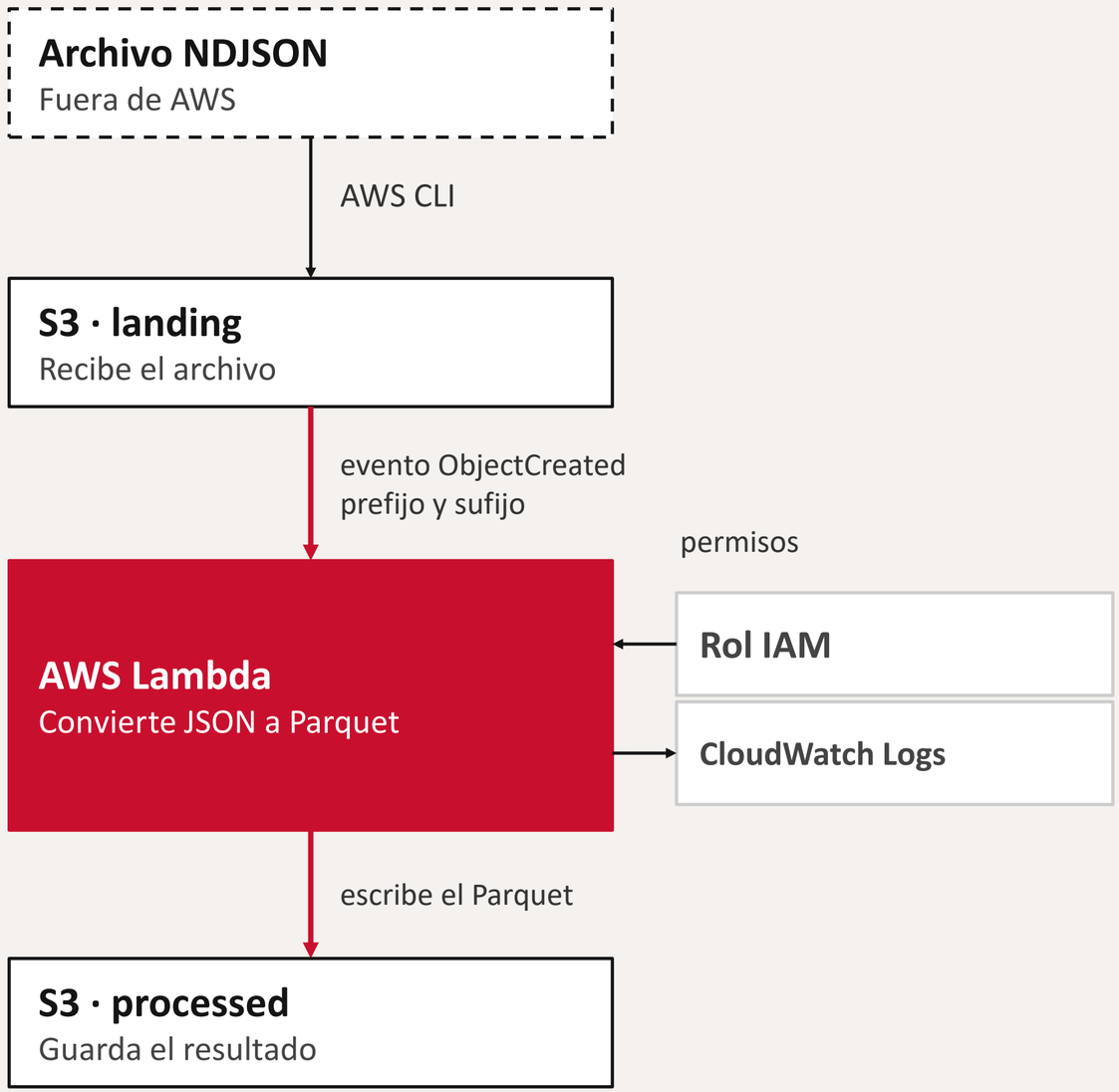
Compartido y con versionado; un bloqueo evita dos apply a la vez.

04

Del código al pipeline

El pipeline que vamos a leer

Dos buckets, un rol y una Lambda que reacciona reacciona al evento.



S3, IAM y Lambda en el código

AMAZON S3

Dos buckets: landing y processed

Prefijos y formatos: JSON entra, Parquet sale. sale.

AWS IAM

Un rol con mínimo privilegio

Solo lee de landing y escribe en processed.

AWS LAMBDA

Un handler de unas 25 líneas

Timeout, memoria y el evento que la dispara. dispara.

El rol de la Lambda

La política dice qué puede hacer la Lambda. Lambda. ¿Qué pasaría con s3:*?

```
# archivo: iam.tf
data "aws_iam_policy_document" "lambda" {
  statement {
    actions   = ["s3:GetObject"]
    resources = ["${aws_s3_bucket.landing.arn}/*"]
  }
  statement {
    actions   = ["s3:PutObject"]
    resources = ["${aws_s3_bucket.out.arn}/*"]
  }
}
```

Anatomía de una Lambda

EVENTO

Llega un evento evento

S3 avisa que hay un archivo

→

HANDLER

Corre tu código

Una función que recibe el recibe el evento

→

ROL

Con permisos mínimos

Solo lo que el rol permite permite

→

LOGS

Todo queda escrito

Cada print va a CloudWatch

El handler

Llega un evento, se lee el NDJSON y se escribe el Parquet.

```
# archivo: handler.py
def handler(event, context):
    info = event["Records"][0]["s3"]
    bucket = info["bucket"]["name"]
    key = info["object"]["key"]
    print(f"Evento recibido: {key}")

    tabla = leer_ndjson(bucket, key)
    salida = escribir_parquet(tabla, key)
    return {"parquet": salida}
```

Quién puede invocar invocar la Lambda

El rol dice qué puede hacer la Lambda. Este Este permiso dice quién puede llamarla.

```
# archivo: trigger.tf
resource "aws_lambda_permission" "s3" {
  statement_id  = "AllowS3Invoke"
  action        = "lambda:InvokeFunction"
  function_name = aws_lambda_function.fn.arn
  principal     = "s3.amazonaws.com"
  source_arn    = aws_s3_bucket.landing.arn
}
```

El evento de S3

El filtro de prefijo y sufijo decide qué archivos archivos disparan la Lambda.

```
# archivo: trigger.tf (continuación)
resource "aws_s3_bucket_notification" "n" {
  bucket = aws_s3_bucket.landing.id

  lambda_function {
    lambda_function_arn = aws_lambda_function.fn.arn
    events        = ["s3:ObjectCreated:*"]
    filter_prefix = "entrada/"
    filter_suffix = ".ndjson"
  }

  depends_on = [aws_lambda_permission.s3]
}
```

Plan del laboratorio

86–90	90–97	97–107	107–110
Paso 1 **Revisar el repo** Estructura, perfil y plan	**Paso 2** **Apply** Despliega el pipeline	**Paso 3** **Subir y verificar** Evento, Parquet y logs	**Paso 4** **Destroy** Limpieza y comprobación

Antes de aplicar, lee el plan

- Hoy esperamos solo **+**: todo se crea desde cero.
- Mira el resumen final: cuántos recursos se agregan, cambian o se destruyen.
- Si destruye algo que no esperabas, no apliques y pregunta.

Despliega el pipeline

Tres comandos y Terraform te devuelve los los nombres de tus recursos.

```
# terminal: carpeta del laboratorio
terraform init
terraform plan
terraform apply

# al terminar, Terraform muestra:
# bucket_landing   = "bootcamp-landing-..."
# bucket_processed = "bootcamp-processed-..."
# lambda_name      = "bootcamp-etl"
```

Sube el archivo y observa

Subir el NDJSON dispara la Lambda. Nadie la
Nadie la invoca.

```
# terminal: subir y verificar
aws s3 cp datos/muestra.ndjson \
  s3://<landing>/entrada/ \
  --profile bootcamp

aws s3 ls s3://<processed>/ \
  --profile bootcamp --human-readable
aws logs tail /aws/lambda/<nombre> \
  --profile bootcamp --since 5m
```

Qué debes ver

- Un archivo .parquet en el bucket processed.
- En los logs, la línea «Evento recibido» con el nombre de tu archivo.
- La Lambda corrió sola: tú solo subiste el archivo.

Destruye el entorno

Terraform elimina lo que creó. Verificamos
Verificamos que no quede nada.

```
# terminal: limpieza
terraform destroy

# comprobar que no queda nada
terraform state list
aws s3 ls --profile bootcamp
aws logs describe-log-groups \
  --log-group-name-prefix /aws/lambda/<nombre> \
  --profile bootcamp
```

Para la sesión 2

- ¿Qué cambia si en lugar de un archivo llegan miles?
- ¿Cómo catalogamos los datos y los consultamos con SQL?
- ¿Cómo coordinamos varias etapas de un mismo flujo?
- **Tarea:** que terraform version y aws sts get-caller-identity funcionen.

05 · Cierre

Adelanto: parte de una spec

En la sesión 2, Claude Code generará la infraestructura a partir de una spec como esta.

```
# archivo: spec/00-contexto.md
## Objetivo
Convertir NDJSON en Parquet cuando un
archivo llega a S3.

## Restricciones
- Una sola región
- Rol con mínimo privilegio
- Buckets landing y processed
```

Lo que te llevas de la sesión 1

01 Trabajas con un IAM user y un perfil de CLI, no con root.

02 Todo en AWS es una API, y Terraform la llama con código.

03 Un evento en S3 dispara la Lambda: tu primer ETL ya reacciona.

Referencias

Amazon Web Services. (s. f.-a). *AWS Certified Data Engineer – Associate (DEA-C01): Exam guide*. https://docs.aws.amazon.com/aws-certification/latest/data-engineer-associate-01/data-engineer-associate-01.html

Amazon Web Services. (s. f.-b). *AWS Lambda managed layers*. https://aws-sdk-pandas.readthedocs.io/en/stable/layers.html

Amazon Web Services. (s. f.-c). *Choosing an AWS Free Tier plan*. https://docs.aws.amazon.com/awsaccountbilling/latest/aboutv2/free-tier-plans.html

Amazon Web Services. (s. f.-d). *Process Amazon S3 event notifications with Lambda*. https://docs.aws.amazon.com/lambda/latest/dg/with-s3.html https://docs.aws.amazon.com/lambda/latest/dg/with-s3.html

Amazon Web Services. (s. f.-e). *Security best practices in IAM*. https://docs.aws.amazon.com/IAM/latest/UserGuide/best-practices.html

HashiCorp. (s. f.-a). *Backend type: s3*. https://developer.hashicorp.com/terraform/language/backend/s3

HashiCorp. (s. f.-b). *Terraform AWS provider documentation*. https://registry.terraform.io/providers/hashicorp/aws/latest/docs

esan UNIVERSITY

Gracias

¿Preguntas?

A

Anexos

Material de respaldo para preguntas

Buenas prácticas en el código

- Etiquetas en los recursos para identificar el laboratorio.
- Retención corta de logs: el log group lo declara Terraform.
- Block Public Access y cifrado por defecto en los buckets.
- force_destroy en los buckets del laboratorio, para que destroy no falle.

Plan B: invocar a mano

Si el evento no dispara, invoca la Lambda con un evento de prueba.

```
# terminal: respaldo si el evento no dispara
aws lambda invoke \
  --function-name <nombre> \
  --payload file://evento.json \
  --cli-binary-format raw-in-base64-out \
  --profile bootcamp salida.json

# revisar la notificación del bucket
aws s3api get-bucket-notification-configuration \
  --bucket <landing> --profile bootcamp
```

Errores frecuentes

- AccessDenied: revisa el perfil, la región y los permisos del usuario.
- Perfil o región equivocados: comprueba aws configure list --profile bootcamp.
- Claves mal copiadas: vuelve a configurar el perfil.
- PATH en Windows: cierra y abre la terminal después de instalar.

Qué nunca se sube a Git

- *.tfstate: el estado puede contener datos sensibles.
- .terraform/: descargas del provider, pesadas e innecesarias.
- *.tfvars: ahí pueden acabar el perfil o variables propias.
- .env: variables de entorno y claves.

El mapa de DEA-C01

DOMINIO	PESO	SESIÓN 2	HOY
Ingesta y transformación	34 %	Glue Job	**S3 y Lambda**
Gestión de almacenes	26 %	Catálogo y Athena	**Formatos en S3**
Operaciones y soporte	22 %	Step Functions	**Logs y limpieza**
Seguridad y gobernanza	18 %	Roles por servicio	**IAM y access keys**

Si te sobra tiempo

- Sube un archivo con otro sufijo y comprueba que el filtro no dispara la Lambda.
- Lambda.
- Sube un segundo archivo y compara los logs de las dos ejecuciones.
- Cambia el filtro en Terraform, revisa el plan y vuelve a aplicar.

Checkpoint tipo examen

- ¿Qué gana si una política permite una acción y otra la deniega de forma explícita?
- ¿Qué archivo usa Terraform para saber qué recursos ya creó?
- ¿Qué símbolo del plan indica destruir y recrear un recurso?
- ¿Por qué S3 necesita un permiso para invocar la Lambda, además del rol?

Drift: cambia la consola

Si alguien cambia algo a mano, el plan lo detecta.

```
# terminal: provocar drift (hora de consultas)
# 1. añade una etiqueta al bucket en la consola
# 2. compara el código con la realidad
terraform plan

# el bucket aparece marcado con ~ (cambia)
# 3. vuelve al estado que dice el código
terraform apply
```

Backend remoto en S3 en S3

El bucket debe existir antes y tener versionado. El bloqueo ya no usa DynamoDB. DynamoDB.

```
# archivo: backend.tf (ejemplo)
terraform {
  backend "s3" {
    bucket       = "<bucket-del-state>"
    key          = "bootcamp/terraform.tfstate"
    region       = "<región>"
    use_lockfile = true
  }
}
```

Qué es un módulo

- Un módulo es una carpeta con recursos que se reutilizan.
- El módulo raíz llama a otros con un bloque module.
- Hoy todo el pipeline vive en el módulo raíz.